Nadia El Jokkari'

Coding 2 pdf

Week 1 :

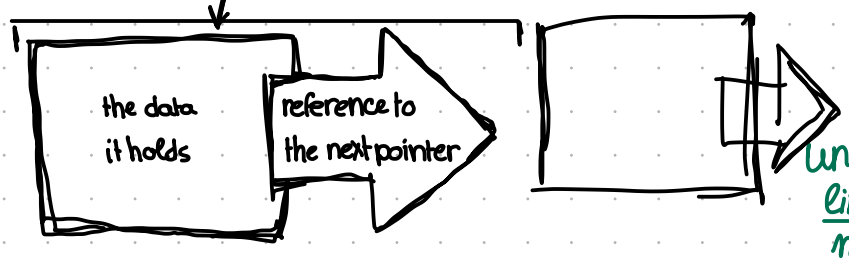
→ stacks & queues are linear data structures

Stack
→ FILO push() & pop()

Queue
→ LIFO enqueue() & dequeue()

Week 2 :

Linked lists → another linear data structure
→ consists of nodes:



Unlike Python's built-in lists, linked lists must be implemented manually using classes

Week 3 Linear data structure
Introduction to set theory & graph theory

set theory → a group of unique items/values
NO 2 ITEMS CAN BE THE SAME
(re-visit Venn diagrams ∩ = & ∪ = | with intersections (∩) & unions (∪))

graph theory → how nodes are connected by lines

example:

set theory → list of people

graph theory → map of connections between people

Week 4 Set theory & graph theory

Adjency list → a data structure to represent a graph where each node in the graph stores a list of its neighbouring vertices
↳ another word for Node

Adjency matrix* → where each cell on index (i,j) stores an information about the edge from vertex i to vertex j
↳ 2D array

Week 5 - Graph Theory

The purpose of graphs is to encode relations on a finite set, specifically, Each vertex (NODE) on a graph is represented by a point — the graph is basically what connects them.

↳ I can basically use graphs to create a list of adjacencies → which NODE is connected to what, what there is in between each node, what connects them, etc

↳ a way to map data

Week 6 + 7 : Linear Algebra

Vectors → one dimensional array of lists
vector additions & multiplications & substractions work

$V1 = [10, 11, 12, 13]$

$V2 = [1, 2, 3, 4]$

$V3 = V1 + V2$

$= [11, 13, 15, 17]$

Basic linear transformation → function that map a vector from one space to another

$y = A \cdot x$ → an example of a basic linear transformation is scaling

Week 8 - Numpy + intro to Task 2

dot product → mathematical operation that takes 2 vectors & combines them to produce a singular scalar value

Task*1 A Priority Queue for people's names

How (I understood the way) it works with the most important functions

def __init(self, data)

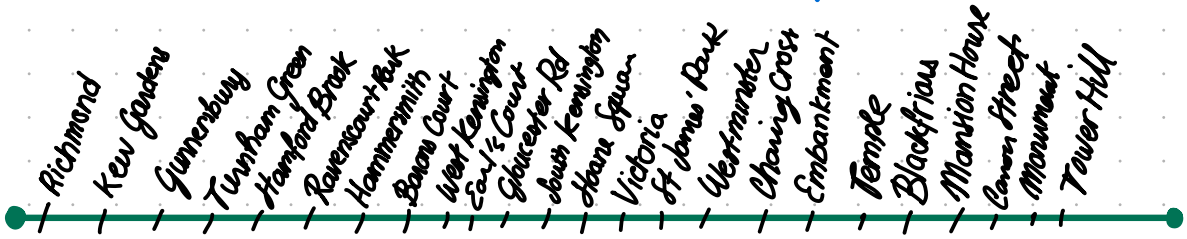
initialising the queue

def peek(self)
returns highest priority element

def add(self, data)
- adds element to the list
- sorts it by priority

How do I develop this concept as a task further?

The 1st task's concept is basically a self implemented sorting queue which rearranges the given data based on priority.
We were given another peice of code from the same tasks that uses this concept to sort TFL's train lines.
This is when I got the idea to create some sort of tube map route display



district line

Richmond ↔ Tower Hill

What will this code do?

You state from where on this line you are departing

& where you want to go.

The code will show you the route of stations ahead of you.

using Node classes, linked lists, stacks, queues, & set theory

I will try to use the function that organises by priority as a way to classify the journey by distance

Through this I decided to also implement Node classes & apply graph theory

Task 1 → What would each function be used for?

STATIONS represented by nodes in a graph / linked list

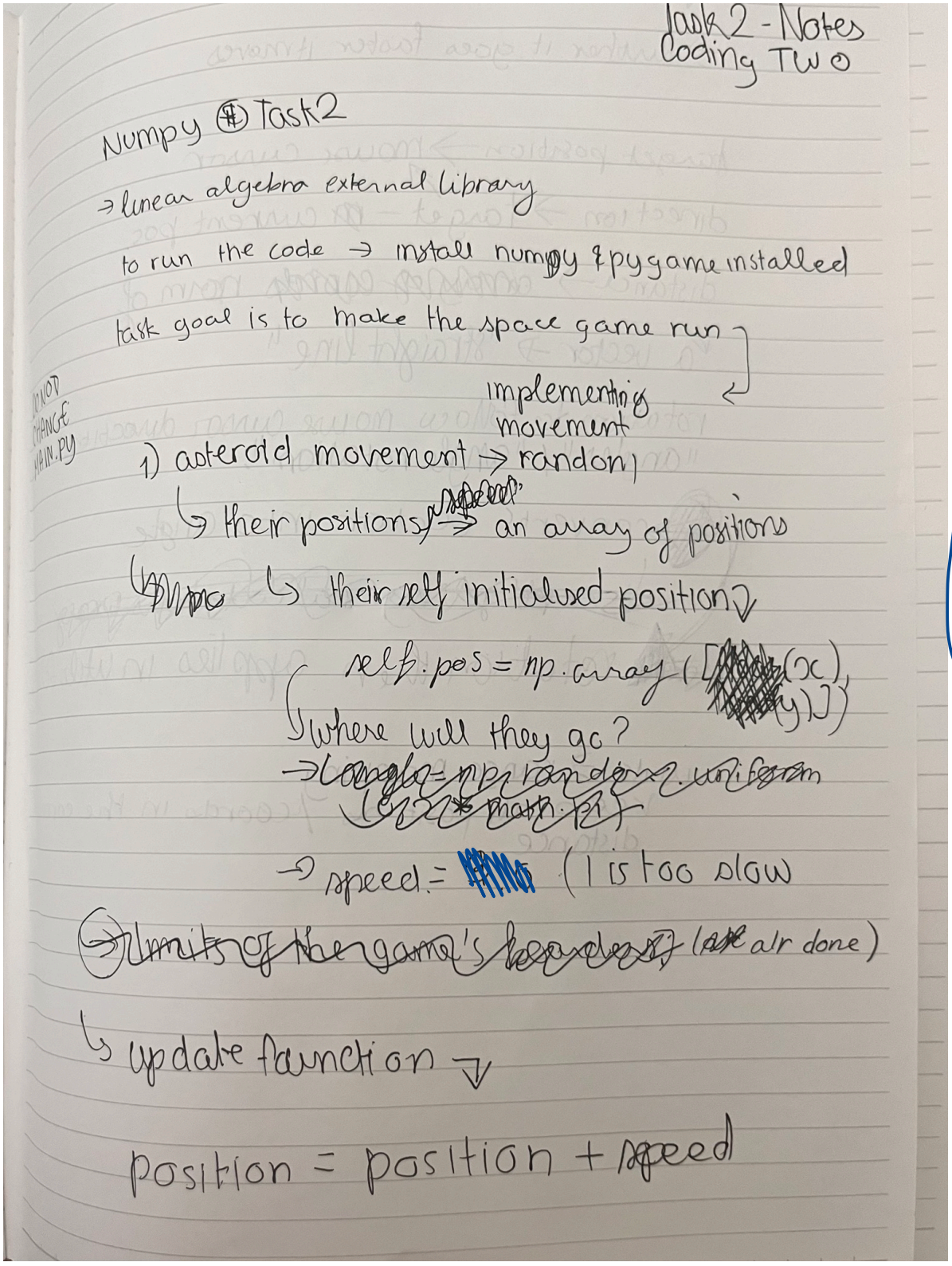
GRAPH adjacency with distances between consecutive stations

Priority-Queue adapted from the uploaded code to order distance of departure
setting this up 1st

SET THEORY visited set to avoid revisiting nodes

LINKED-LIST the district line is a linked list of nodes = stations

TASK 2- Asteroid dodging game



Task 2 - Notes
Coding TWO

Numpy # Task2

→ linear algebra external library

to run the code → install numpy & pygame installed

task goal is to make the space game run

implementing movement

DO NOT CHANGE MAIN.PY

1) asteroid movement → random

↳ their positions → an array of positions

↳ their self initialised position

self.pos = np.array([(x), (y)])

where will they go?

→ angle = np.random.uniform(0, 2 * math.pi)

→ speed = 1 (1 is too slow

→ ~~limits of the game's boarders~~ (already done)

↳ update function

position = position + speed

part 1 : asteroids

↓

making them move randomly across the screen

→ speed needs to be constant

→ no accumulation

→ current asteroid position needs to update constantly

ship position + utils

ship cursor

→ further it goes faster it moves

target position → mouse cursor

direction → target − current pos.

distance → norm of

↖ a vector ⇒ "straight line"

rotation to follow mouse cursor direction:
"angles" / "angle rotation":

converts vector into angle

rotation then applies in utils file

utils ⇒ range mapping
keep all positions / coords in the correct distance

Task 2 - process

Asteroid.py

- a full circle = $2\pi \simeq 6.28$
  - ↳ direction for an asteroid = random number within that range
    - → same for speed
- directions are usually singular & linear, must turn the "angle" into another value so it looks like its rotating even though its not
  - ↳ using cos & sin
- multiplying by velocity & speed.
  - cos → left & right
  - sin → up & down

updating position:

position = position + position's velocity

Ship.py

position following the mouse → x, y coordinate system that follows/copies the cursor's coords every single frame

atan2 - reverse of cos & sin taking on x & y gap & working backwards:

mouseX − shipX
mouseY − shipY

collision check
→ calculate straight line distance between the ship & asteroid w/pythagoras

$$\text{distance} = \sqrt{(\text{asteroidX} - \text{shipX})^2 + (\text{asteroidY} + \text{shipY})^2}$$

↳ if distance < to this
collision == True

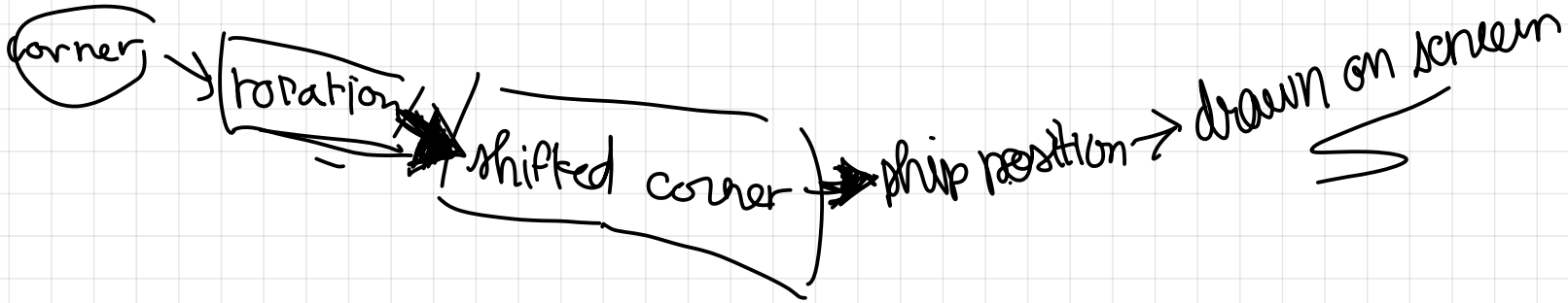
there is a gap between ship & cursor

utils.py

Rotation formula applied

new x = x * cos(angle) - y * sin(angle)

new y = x * sin(angle) + y * cos(angle)



The whole game comes down to 3 bits of maths

1 — cos + sin → x & y movements so asteroids can fly in any direction

2 — atan2 reverses it and takes the gap & works out the angle so the ship always faces the cursors

3 — pythagoras measures the straight line distance between 2 points — collision detection

Task 3 - Maze algorithm

Dijkstra's algorithm

Coding Two - Maze Algorithm

- the way a maze works:
  puzzle to get through a range of paths
  ↳ to get through the maze, finding the longest path

Graph class ↙
- adjacency list: who are my neighbours?
  ↳ connections between nodes
= incidence matrix → which roads touch which city

Giving every destination "best distance"
↳ implementation of a walk class
↳ demonstrates which distance to walk
↳ create a vertex to remember each path
↳ create a list of paths not walked yet
↳ start with the starting path (distance=0)

main loop to keep going until finished:
while the paths remain unvisited,
- the current distance gets added to the list of visited paths
- remove the path from unvisited paths

create a function to give shortcuts
↳ when a shorter distance is found
↓ if the new distance is < than current

The whole point of this code is a sorting system based on distances

graph.py

graph = collection of points

↳ for every point there is a connection
= adjacency list

↳ adjacency matrix = a grid

the walk:
finding shortest distance

if current distance + edge cost < recorded distance

tracing path back

a → none (starting point)

b → a

c → c

d → b

@ the end you start @ d & follow the previous pointers backwards

d → b → a → none

reverse = a b d

↳ shortest path